KVSwitch: Accelerating Distributed LLM Inference with In-Network Prefix-Aware Routing

Noppanat Wadlom
National University of Singapore

Satis Kumar Permal
National University of Singapore

Abstract

Distributed LLM serving systems increasingly rely on prefix caching to reduce redundant computation and improve inference latency. However, existing prefix-aware routing mechanisms are commonly implemented in centralized layer-7 routers, introducing non-negligible overhead that increases time to first token (TTFT). We present KVSwitch, a system that offloads prefix-aware routing into programmable network switches. A client SDK tokenizes each prompt, computes a cumulative prefix hash chain, and embeds it in a compact shim header that switches process directly in the data plane. A cache-event-driven SDN controller keeps switch rules synchronized with worker cache state and load, driving per-prefix weighted ECMP that jointly balances cache locality and worker load under TCAM capacity constraints. On a BMv2-based prototype with a Mininet-emulated two-tier Clos topology and an LLM serving simulator, KVSwitch reduces median TTFT by up to 27% and tail TTFT by up to 76% relative to a state-of-the-art layer-7 prefix-aware router.[1]

1 Introduction

Recent years have seen a rapid surge in the adoption of large language models (LLMs) for both interactive user applications [3, 4, 7] and offline analytics [16, 24, 26]. To manage the intensive computational demands of autoregressive text generation, modern serving systems rely heavily on the KV cache [25], which stores intermediate attention states in GPU memory to avoid redundant computation. Because multiple user requests often share common prompt prefixes (e.g., system instructions or few-shot examples), systems organize these KV caches into radix trees [23] or chunked hash maps [20] to enable cross-request prefix sharing [28]. This prefix caching dramatically reduces the time to first token (TTFT), a critical metric for service-level agreements (SLAs).

To serve millions of concurrent users, cloud providers deploy distributed LLM inference clusters [2, 10, 11]. To maximize cache hit rates across these distributed GPU nodes, incoming requests must be routed to the specific server holding the longest matching prefix cache. However, state-of-the-art (SOTA) serving systems currently rely on centralized, Layer-7 (L7) routers to make these routing decisions [8, 9, 21, 28]. These software-based load balancers introduce significant latency overheads, terminating TCP connections, parsing HTTP/RPC payloads, tokenizing prompts, and proxying requests through the host networking stack, which ultimately bottlenecks throughput and inflates TTFT.

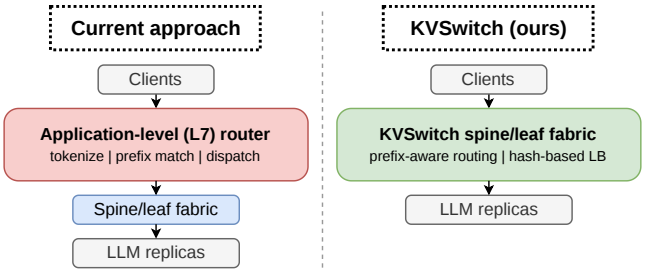


Figure 1: Prefix-aware routing in distributed LLM serving. Traditional systems (left) place the decision in a centralized L7 router; KVSwitch (right) offloads it into the spine/leaf fabric, eliminating L7 overheads.

How do we eliminate such L7 routing overhead while still enabling efficient prefix-aware routing? Fortunately, in the era of software-defined networking (SDN), the adoption of programmable data planes [14] has opened up new opportunities to offload complex packet processing tasks from software to hardware. However, putting prefix-aware routing logic directly into the data plane poses significant challenges. First, prefix-aware routing requires information about the cache prefixes in each LLM replica. An L7 router can easily track this information by keeping records of prompt prefixes dispatched to each server, but network switches lack this visibility and require a mechanism to synchronize cache state information from the LLM engine. Second, naive prefix-aware routing can lead to load imbalance, as requests with the same prefixes

[1] Our code is available at https://github.com/kaiitunnz/kvswitch.

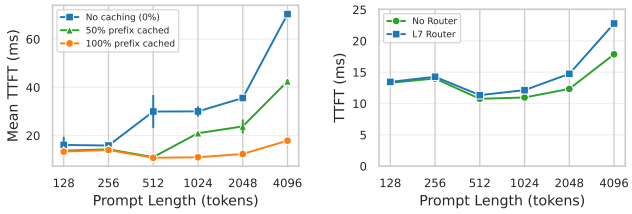
tend to concentrate on a few replicas, while others remain underutilized. SOTA routing algorithms [8, 28] employ simple heuristics to balance this cache-load tension, but implementing such logic in the data plane is non-trivial. Finally, programmable switches have limited memory resources for storing routing rules, and the number of unique prefixes can grow rapidly with workload diversity, necessitating efficient rule management strategies.

To address the above challenges and enable in-network prefix-aware routing, we present KVSwitch, a hardware-software co-designed system that leverages programmable network switches to perform prefix-aware routing at line rate (Figure 1). Inspired by the chunk-based prefix caching in vLLM [20], KVSwitch maps tokenized prompt chunks into hierarchical hashes, allowing switch hardware to perform hierarchical prefix matching using ternary content-addressable memory (TCAM). By maintaining cache state synchronization via a lightweight SDN control plane integrated with the LLM engine's internal cache manager, KVSwitch configures the data plane to route and load-balance requests via per-prefix ECMP at hardware speed, bypassing L7 software overheads. Our comprehensive evaluation using a BMv2-based prototype and a Mininet-emulated Clos topology demonstrates that KVSwitch can achieve up to 27% lower median TTFT and 76% lower tail TTFT compared to traditional L7 prefix-aware routing, while effectively balancing load across replicas under TCAM constraints. In summary, our work makes the following contributions:

- We identify the critical bottleneck of L7 routing overhead in distributed LLM inference systems and propose offloading prefix-aware routing to programmable switches.
- We design KVSwitch, a novel system that integrates in-network prefix matching and a cache-state-driven SDN controller to enable prefix-aware routing in the data plane.
- We implement a prototype of KVSwitch using the BMv2 software switch and evaluate its performance in a Mininet-based Clos topology with an LLM serving simulator, demonstrating significant reductions in TTFT compared to traditional L7 routing approaches.

2 Background and Motivation

LLM inference and KV cache. LLM inference is an autoregressive process, generating text token by token. To accelerate this computationally intensive workload, modern inference engines maintain a KV cache [25] in GPU memory, which stores the intermediate attention states of previously processed tokens. Because multiple user requests often share common prompt prefixes, such as identical system instructions or few-shot examples, their corresponding KV cache entries can be shared to avoid redundant computation. Systems typically organize this cached state into radix trees [23] or block-level hash maps [20], allowing incoming requests to perform LPM against the cached states, thereby significantly reducing the TTFT and improving overall serving throughput, as illustrated in Figure 2a.



(a) TTFT vs prompt length (b) L7 router overhead

Figure 2: (a) TTFT reductions from prefix caching across different prompt lengths. (b) Breakdown of latency overheads introduced by a centralized L7 prefix-aware router.

L7 prefix-aware routing. In a distributed deployment [2, 8, 10, 11], different LLM replicas, or, interchangeably, LLM workers, house different prefix caches based on their request histories. To maximize cache reuse, current SOTA serving systems employ an L7 prefix-aware router. When a request arrives, this centralized router must terminate the TCP connection, parse the HTTP/RPC payload, tokenize the input prompt, traverse its software-maintained cache directory to identify the optimal backend server, and finally proxy the request. This pipeline introduces substantial latency overheads due to kernel-to-user-space context switching, socket I/O, and serialization, as well as the computational cost of tokenization and cache directory traversal. To illustrate, we implement a simple L7 prefix-aware router in Python and measure the lower bound of the L7 routing overhead as the wall-clock time of application-layer processing, excluding the network stack overheads. As shown in Figure 2b, even this minimal implementation incurs an overhead of 4.9 ms on average at 4096 tokens, increasing the TTFT by up to 27.5% compared to a direct routing baseline. Furthermore, as request rates scale, this L7 router can become a throughput bottleneck.

In-network load balancing. Network routing involves forwarding packets along the paths towards their destination. For efficiency, network routers and switches often employ load-balancing techniques to distribute traffic across multiple paths. The oldest and most widely deployed primitive is equal-cost multi-path (ECMP) [17]: when multiple shortest paths to a destination exist, a switch hashes the 5-tuple of each packet to deterministically spread flows across them. Due to its simplicity and effectiveness, ECMP is the default scheme in modern switch fabrics; however, it treats every packet as opaque and is insensitive to either path congestion or application-level load. Congestion-aware schemes address the first limitation by incorporating path-level telemetry: CONGA [13] measures per-leaf round-trip utilization and biases ECMP accordingly, while WCMP [29] assigns

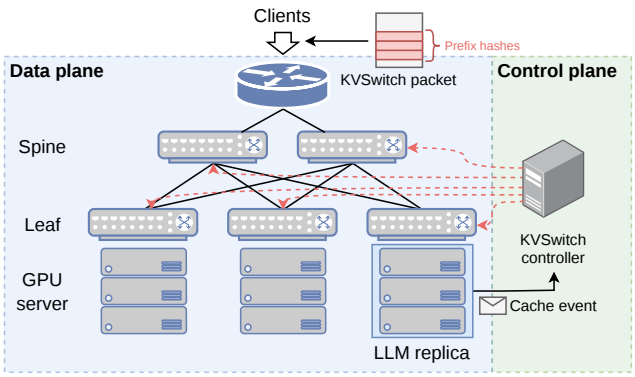


Figure 3: KVSwitch overview. The client SDK computes prompt-prefix hashes and encodes them in a shim header. Switches perform hierarchical prefix matching and per-prefix ECMP routing. The SDN controller maintains TCAM rules based on cache events from workers.

bandwidth-proportional weights to compensate for asymmetric fabrics. The advent of SDN and programmable data planes [14] in turn enables *application-aware* in-network routing, in which switches parse and match on application-level headers or keys at line rate. AppSwitch [15] and LBAS [27], for instance, perform per-request, server-aware load balancing for key-value and web services entirely within the fabric. While these schemes are application-aware, they target workloads with simple key-based semantics and are not designed for prefix-aware routing in distributed LLM inference.

3 System Design

3.1 Overview

KVSwitch leverages programmable network switches to perform prefix-aware routing, eliminating the overhead of centralized L7 routers. The system consists of three main components: (1) a client SDK that computes prompt-prefix hashes and encodes them in a compact header; (2) a programmable data plane that performs hierarchical prefix matching and per-prefix ECMP routing; and (3) an SDN controller that maintains TCAM rules based on cache events from LLM workers. The overall architecture is illustrated in Figure 3.

Client SDK. To completely bypass L7 routing, KVSwitch requires the client requests to carry metadata about the prompt prefixes. To this end, KVSwitch delegates the task to the client SDK, which computes the prefix hashes and encodes them into a shim header that can be processed by the data plane. Specifically, it tokenizes the input prompt, computes hierarchical hashes for each prefix chunk, and constructs a lightweight *shim header* that encapsulates this information. This header is then prepended to the application payload before being sent to the network.

Programmable data plane. The data plane is the core component of KVSwitch, where the prefix-aware routing logic is implemented. It uses TCAM to perform hierarchical prefix matching based on the hashes in the shim header. The data plane also implements per-prefix ECMP routing to balance load across LLM workers while maximizing cache reuse. In conventional data center topologies where switches are organized in tiers, the prefix rules can be installed such that the higher-tier switches perform coarse-grained matching on shorter prefixes, while the lower-tier switches perform finer-grained matching on longer prefixes, effectively utilizing TCAM resources.

Cache-event-driven control plane. Workers report KV cache state changes (prefix allocations, evictions, and load updates) to an SDN controller via asynchronous events. The controller translates these events into TCAM rule installations and ECMP weight adjustments, keeping the data-plane forwarding state tightly synchronized with the cache state. A joint cache-load optimization governs the weight computation to balance between cache efficiency and worker load.

Scope. KVSwitch targets *single-operator, private data center* deployments where all switches, workers, and the control plane are managed by a single trusted entity; multi-tenant isolation is out of scope. Clients are assumed to run the *KVSwitch SDK*, which performs lightweight CPU-only tokenization and hashing, consistent with enterprise Model-as-a-Service deployments where client libraries are standard. The design assumes a two-tier leaf-spine (Clos) fabric, which is common in modern data centers, but the principles can be adapted to other topologies. Finally, KVSwitch is most effective for *prefix-sharing workloads*, where a significant fraction of requests share common prompt prefixes, as is typical in LLM inference scenarios with system instructions and few-shot examples.

3.2 Shim Header

KVSwitch encodes routing hints in a 20-byte *shim header* inserted between the UDP header and the application payload. The switch parser recognizes the header by a designated UDP destination port and extracts the prefix hash fields for matching. The header format is illustrated in Figure 4.

The KVSwitch header consists of a 4-byte metadata field and four 4-byte hash fields. The metadata field includes a 4-bit version number, a 4-bit prefix length field (specifying how many of the hash fields are valid), an 8-bit flags field for future extensions, and a 16-bit request ID for tracking. The hash fields contain the hierarchical prefix hashes computed by the client SDK from the prompt's token sequence as follows. The client tokenizes the prompt and partitions the resulting token IDs into chunks of C tokens. Starting from a fixed root hash, each chunk is concatenated with its parent hash and fed through HMAC-SHA256; the output becomes both the hash value h_i and the parent for the next chunk:

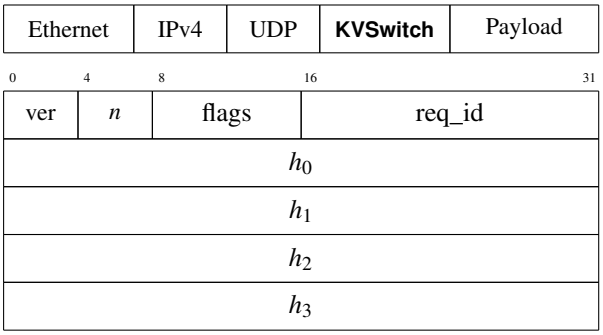


Figure 4: KVSwitch packet layout (top) and shim header format (bottom). The 20-byte shim header, containing the prefix hashes, is inserted between the UDP header and the application payload.

$$h_i = \text{HMAC-SHA256}\big(k,\ h_{i-1} \,\|\, \text{pack}(\text{tokens}[iC\,..\,(i{+}1)C))\big),$$

where h_{-1} is a fixed 256-bit zero vector and k is a shared HMAC key. The cumulative chaining ensures that h_i is a function of all tokens in chunks 0 through i, mirroring the prefix semantics of the KV cache: two prompts sharing the first $i{+}1$ chunks produce identical $h_0, \ldots, h_i$. Each 256-bit digest is truncated to 32 bits to keep the shim header compact enough for in-switch parsing; the header supports up to four hash slots.

3.3 Data Plane

The data plane implements a two-tier match-action pipeline in P4 that runs on every switch in the leaf-spine fabric. Spine switches and leaf switches execute the same program; only the table entries installed by the controller differ.

At the spine tier, a ternary match on a prefix of the hash tuple $(h_0, \ldots, h_{n-1})$ identifies the *prefix ECMP group*, which is a set of weighted leaf destinations that have the prefix cached. At the leaf tier, a finer-grained ternary match on a longer prefix of the same tuple identifies a *leaf prefix ECMP group*, which is a set of weighted local LLM workers. Both levels use an indirection through per-prefix ECMP tables: a prefix match sets a group ID, and a second exact-match table maps (group ID, ECMP bucket) to a physical port. This separation allows the controller to adjust load-balancing weights without modifying the TCAM prefix entries. To prevent correlated ECMP decisions across tiers, the spine and leaf switches compute independent hash-based bucket indices using distinct hash algorithms (e.g., CRC-16 and CRC-32) over the same 5-tuple. If the prefix is not matched at either tier, the packet falls back to the default ECMP routing that balances across the next hops to accommodate unseen prefixes. This also serves traffic during TCAM warm-up, letting the system transition smoothly to per-prefix routing as the controller admits popular prefixes. The ingress pipeline logic is summarized in Algorithm 1.

Algorithm 1: P4 ingress pipeline.

```
Input: Packet with headers hdr, metadata meta
if hdr.kvswitch is valid then
    meta.spine_bucket ← Hash_1(5-tuple)
    meta.leaf_bucket ← Hash_2(5-tuple)
    if leaf_prefix_route(h_0, ..., h_{n-1}) hits then
        look up leaf_prefix_ecmp(group,
          leaf_bucket)
        route to worker
    else if spine_prefix_route(h_0, ..., h_{m-1}) hits
      then
        look up spine_prefix_ecmp(group,
          spine_bucket)
        route to leaf
    else
        if leaf_ecmp(leaf_bucket) misses then
          spine_ecmp(spine_bucket)
    end
end
else if hdr.ipv4 is valid then
    apply IPv4 LPM forwarding
end
```

3.4 Control Plane

The SDN controller maintains a real-time view of distributed KV cache state by consuming three types of asynchronous cache events from the workers:

- `alloc`: a new prefix has been cached after serving a request.
- `evict`: a previously cached prefix has been evicted from the cache due to capacity constraints.
- `queue_update`: a worker's load has changed, as indicated by its request queue length or batched token budget usage.

TCAM admission. Not every observed prefix warrants a TCAM rule. The controller employs a frequency-threshold policy: a prefix is admitted only after it has been observed at least K times within a sliding window of T seconds. This filters transient or one-off prefixes while ensuring that popular prefixes are installed quickly. When the TCAM reaches capacity, the least-recently-used (LRU) rule is evicted to make room for new entries.

Hierarchical rule management. The controller maintains independent TCAM managers per tier: one for spine rules (keyed on the first m hashes) shared across all spines, and one per leaf switch (keyed on all n hashes). When a prefix is admitted, the controller programs a *per-prefix ECMP group* at the appropriate tier: spine ECMP groups distribute across

Algorithm 2: SDN controller event handler.

Input: Cache event e from worker w

if $e.type = \texttt{queue_update}$ **then**
  update $load[w] \leftarrow e.load$
  schedule coalesced ECMP reconciliation
else if $e.type = \texttt{alloc}$ **then**
  $locations[prefix] \mathrel{+}= \{w\}$
  $hits \leftarrow$ record_observation($prefix$)
  if $hits \geq K$ **then**
    install spine prefix ECMP group
    install leaf prefix ECMP group(s)
  end
else if $e.type = \texttt{evict}$ **then**
  $locations[prefix] \mathrel{-}= \{w\}$
  if $locations[prefix] = \emptyset$ **then** delete prefix rules
  else reweight ECMP group
end

leaves; leaf ECMP groups distribute across local workers. Algorithm 2 outlines the event handling logic.

Per-prefix ECMP weighting. Prefix-aware routing creates a tension between cache efficiency and load balancing: routing all requests with the same prefix to the same worker maximizes cache hits but concentrates load, causing queueing that inflates TTFT. To navigate this tradeoff, each per-prefix ECMP group distributes traffic across all targets that have the prefix cached, weighted by inverse load:

$$w(t) = \frac{1}{1 + load(t)},$$

where $load(t)$ is the current load of target t (either a leaf switch or a worker). The controller maps these weights to ECMP bucket assignments, ensuring that each target receives a share of the B buckets proportional to its weight, with a minimum of one bucket to prevent starvation. As load shifts, the controller reweights the ECMP groups, dynamically balancing cache efficiency and load distribution.

Probe-fraction seeding. When a prefix is first admitted, typically only one leaf or worker has it cached, creating a temporary monopoly. To seed replication, the controller allocates a small *probe fraction* ε of ECMP weight to each uncovered target. This ensures that a trickle of requests reaches uncovered locations; once a probed worker caches the prefix and reports an `alloc` event, the controller replaces the probe weight with a real load-based weight. The mechanism is self-healing: convergence to a multi-replica ECMP group typically occurs within one or two probe requests.

Coalesced update. Under high request rates, `queue_update` events can arrive frequently. Rather than recomputing all ECMP groups on every event, the controller coalesces updates within a configurable interval and performs a single reconciliation pass over all installed prefix rules, amortizing the control-plane overhead.

```
header kvswitch_shim_t {
    bit<4>  version;
    bit<4>  n_chunks;
    bit<8>  flags;
    bit<16> req_id;
    bit<32> h0;
    bit<32> h1;
    bit<32> h2;
    bit<32> h3;
}
```

Listing 1: KVSwitch shim header definition in P4.

4 Implementation

We implement a proof-of-concept prototype of KVSwitch on the BMv2 software switch with a Mininet-emulated Clos topology. The prototype comprises four components: a P4 data plane, an SDN controller, an LLM serving simulator, and a client SDK.

4.1 P4 Data Plane

The data plane is written in $P4_{16}$ targeting the `v1model` architecture. The parser extracts Ethernet, IPv4, and UDP headers; if the UDP destination port matches the designated KVSwitch port, it additionally extracts the shim header (Listing 1). The ingress pipeline contains seven match-action tables: `spine_prefix_route` and `leaf_prefix_route` for hierarchical TCAM matching, `spine_prefix_ecmp` and `leaf_prefix_ecmp` for per-prefix weighted ECMP indirection, `spine_ecmp_select` and `leaf_ecmp_select` for miss-path uniform ECMP fallback, and `ipv4_lpm` for standard IPv4 forwarding on packets that lack the KVSwitch header.

The topology is a parameterized two-tier Clos fabric emulated in Mininet, with configurable numbers of spines, leaves, and workers per leaf. All switches run the same P4 program; only the table entries differ.

4.2 SDN Controller

The SDN controller is implemented in Python using `asyncio` for concurrent event processing. It listens for cache events from workers over UDP and translates them into P4Runtime table operations dispatched through a switch adapter abstraction. The controller maintains per-tier TCAM managers (one shared instance for spine rules, one per leaf switch for leaf rules) that implement the frequency-threshold admission policy and LRU eviction described in Section 3.4. ECMP bucket

```python
class KVSwitchWorkerPlugin:
    def __init__(self, addr, worker_id):
        self.ctrl = ControllerClient(addr)
        self.worker_id = worker_id

    def on_prefill_complete(self, prefix_hashes):
        self.ctrl.send("alloc",
            worker_id=self.worker_id,
            prefix_hashes=prefix_hashes)

    def on_cache_eviction(self, prefix_hashes):
        self.ctrl.send("evict",
            worker_id=self.worker_id,
            prefix_hashes=prefix_hashes)

    def on_schedule_step(self, load_metrics):
        self.ctrl.send("queue_update",
            worker_id=self.worker_id,
            **load_metrics)
```

Listing 2: Example worker-side integration with KVSwitch.

allocations are tracked per switch per prefix, and table updates are issued only when the allocation changes, minimizing control-plane overhead.

4.3 LLM Serving Simulator

Since our evaluation targets routing behavior rather than GPU inference, we use a trace-driven mock worker that simulates vLLM's scheduling and KV cache semantics. Each worker maintains a local KV block cache and computes TTFT as a function of cached and uncached prompt tokens:

$$\text{TTFT} = t_0 + t_c \cdot n_{\text{cached}} + t_u \cdot n_{\text{uncached}},$$

where t_0 is a fixed base latency (model loading overhead, scheduling), and t_c and t_u are the per-token costs for cached and uncached tokens, respectively ($t_c \ll t_u$), all derived from profiling traces on real hardware. Cached tokens still incur work (loading KV blocks, attention over cached state) but skip the expensive prefill computation. The end-to-end latency for a request with n_{out} output tokens is:

$$T_{\text{e2e}} = \text{TTFT} + (n_{\text{out}} - 1) \times \text{TPOT},$$

where TPOT (time per output token) models the autoregressive decoding phase. The simulator enforces capacity limits (`max_num_seqs`, `max_num_batched_tokens`) that mirror vLLM's scheduling constraints; requests exceeding capacity are queued, reproducing the queuing dynamics that make load balancing critical.

Workers report cache events to the SDN controller via fire-and-forget UDP messages. Listing 2 illustrates the example integration points for a production LLM serving engine.

```python
from kvswitch import KVSwitchClient

client = KVSwitchClient(
    host="10.0.0.250",
    port=4789,
    model="meta-llama/Llama-3-8B-Instruct",
    hmac_key=b"shared-secret",
    chunk_size=256,
)
response = client.generate(
    prompt="Summarize the following ...",
    max_tokens=256,
)
```

Listing 3: Example client-side API.

4.4 Client SDK

The client SDK provides a transparent interface for prefix-aware routing. It tokenizes the prompt, computes the cumulative HMAC-SHA256 hash chain (Section 3.2), encodes the shim header, and sends the annotated packet to the KVSwitch service IP. From the application's perspective, the SDK presents the same interface as a standard LLM client library; the prefix hashing is invisible to the caller. Listing 3 shows the example client API.

5 Evaluation

We evaluate KVSwitch to answer three questions:

Q1 How does KVSwitch compare to both prefix-oblivious and prefix-aware routing baselines in terms of TTFT and cache reuse?

Q2 How do KVSwitch's key design choices (per-prefix ECMP load balancing and frequency-threshold rule admission) contribute to its performance?

Q3 How does KVSwitch's performance change under varying workload and system parameters?

5.1 Experimental Setup

Testbed. We emulate a two-tier leaf-spine (Clos) fabric in Mininet, with 2 spines, 2 leaves, 4 LLM workers per leaf (8 workers total), 1 client, and 1 ingress switch. A configurable L7 router is placed in front of the fabric and enabled only for the L7 baselines. All links use a 0.5 ms propagation delay. Switches run the KVSwitch P4 program on BMv2; the SDN controller, the client, the L7 router, and the workers are Python processes communicating over UDP, with the controller and workers kept on a separate out-of-band network. Figure 5 illustrates the topology.

LLM serving simulator. We use the trace-driven mock worker described in Section 4.3, with the coefficients t_0, t_c,

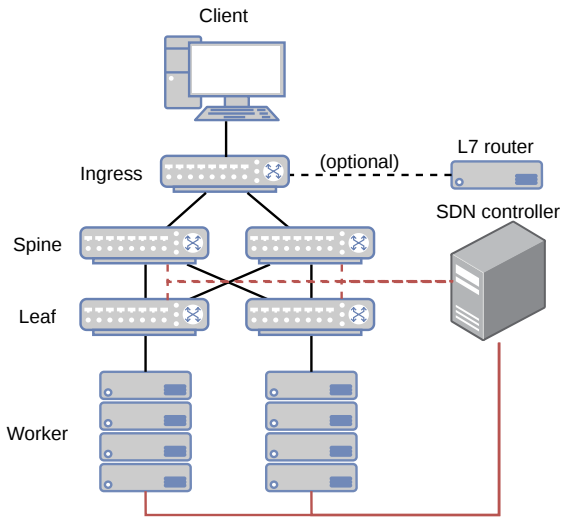


Figure 5: Mininet-emulated topology for evaluation. The L7 router is only enabled for the *L7-RR* and *L7-PA* baselines.

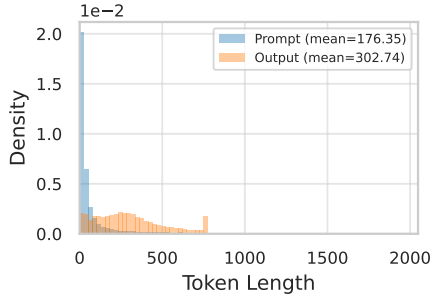


Figure 6: Token length distributions of prompts and outputs in the ShareGPT subset used in our evaluation (mean prompt length 176, mean output length 303 tokens).

and t_u fit by non-negative least squares on inference traces profiled from `meta-llama/Llama-3.2-3B-Instruct` served by vLLM v0.16.0 [6] on a single NVIDIA H100 NVL GPU. TPOT is set to the profile's average per-token decoding time, and the output token count n_{out} is drawn from the ShareGPT dataset.

Workload. We drive the system with prompts sampled from the ShareGPT dataset of real-world conversations between users and ChatGPT [1]; Figure 6 shows the prompt and output token length distributions. To control prefix sharing independently of the raw dataset, we inject a configurable number of synthetic system prompts and assign each sampled user prompt to a prefix group according to a target *prefix sharing ratio* (the fraction of requests that share a prefix with at least one other request). Unless noted otherwise, we use 16 prefix groups, a sharing ratio of 0.8, and Poisson inter-arrival times at a configurable rate in requests per second (rps).

Baselines. We compare KVSwitch against three routing strategies that cover the design space:

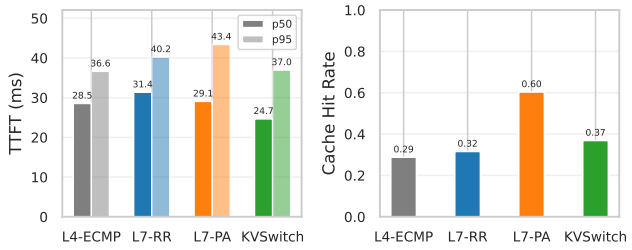


Figure 7: Microbenchmark at 10 rps, 200 requests, 8K KV cache, 0.8 prefix sharing ratio, no warm-up.

- **L4-ECMP**: prefix-oblivious, hash-based L4 load balancing across all workers, with no L7 router.
- **L7-RR**: an L7 router that dispatches requests in round-robin fashion, representing naive L7 routers that ignore prefix state.
- **L7-PA**: a prefix-aware L7 router that performs fine-grained prefix matching against a software-maintained radix tree, modeled on the prefix-aware router in SGLang [28].

All baselines run on the same Mininet topology and the same LLM serving simulator; only the routing strategy differs.

Metrics. We report TTFT at the 50th and 95th percentile, the *cache hit rate* (fraction of input tokens served from a cached prefix), and, where relevant, the *load imbalance factor* (max-to-min of per-worker request counts).

5.2 Microbenchmark

To address Q1, we first isolate the per-request routing cost from queuing effects by running in a controlled low-load regime. Each worker is configured with an 8K-token KV cache; 200 requests are issued at 10 rps with a prefix sharing ratio of 0.8 and no warm-up (Figure 7).

KVSwitch achieves the lowest p50 TTFT among all baselines. Against L4-ECMP, it reduces p50 TTFT by 13.6% and raises the cache hit rate by 8.1 pp. Against L7-RR, it reduces p50/p95 TTFT by 21.4%/8.1%, capturing both the prefix-aware gain and the avoided L7 processing overhead. The comparison with L7-PA is the most informative: L7-PA achieves a noticeably higher cache hit rate (0.60 vs 0.37) thanks to its fine-grained software matching, yet KVSwitch still reduces p50/p95 TTFT by 15.2%/14.8%. In this regime, the per-request overhead of the L7 routing path outweighs the benefit of the higher cache hit rate.

5.3 End-to-End Benchmark

We next evaluate KVSwitch at realistic loads by sweeping the request rate from 10 to 200 rps. We issue 500 requests per run with a prefix sharing ratio of 0.8 and include a warm-up

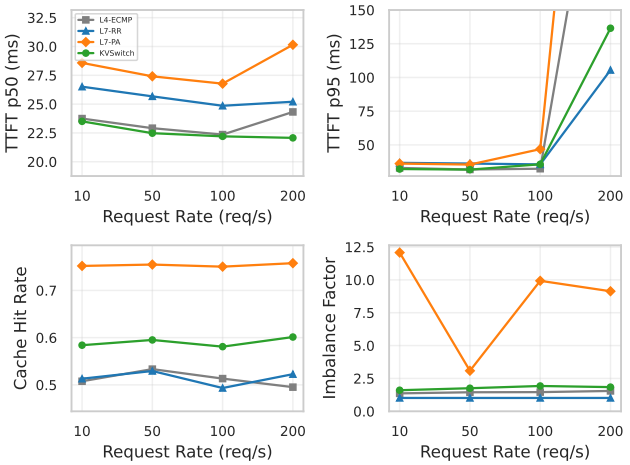


Figure 8: End-to-end benchmark under varying request rates, with 500 requests, 8K KV cache, 0.8 prefix sharing ratio, and warm-up.

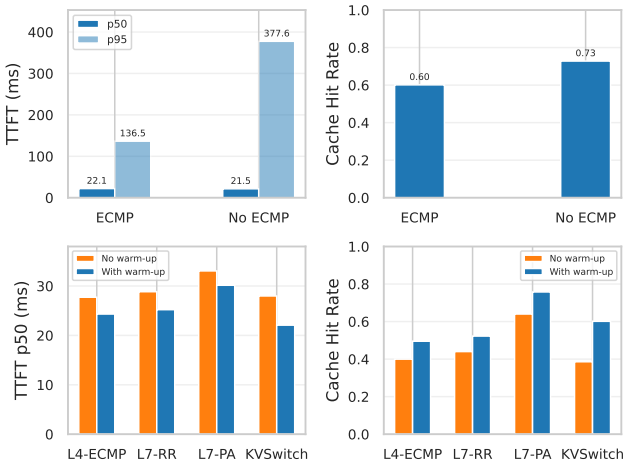


Figure 9: Ablations at 200 rps, 500 requests, 0.8 prefix sharing ratio. Top: per-prefix ECMP vs prefix pinning. Bottom: with vs without warm-up.

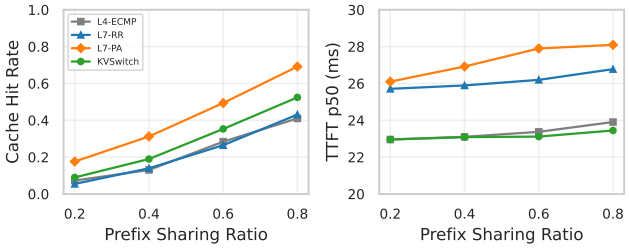


Figure 10: Sensitivity to prefix sharing ratio, at 10 rps, 500 requests, no warm-up.

phase so that popular prefixes are installed in TCAM before measurement begins. Figure 8 shows p50/p95 TTFT, cache hit rate, and load imbalance factor across rates.

First, KVSwitch's advantage over L7-PA widens with load: at 200 rps, KVSwitch reduces p50/p95 TTFT by 26.8%/75.7%. The tail improvement is driven by KVSwitch's lower imbalance factor: L7-PA concentrates traffic on whichever worker currently holds the best-matching prefix, driving its imbalance factor an order of magnitude above the other baselines at 200 rps, whereas KVSwitch's per-prefix weighted ECMP keeps the imbalance factor close to unity by spreading traffic across all replicas that have the prefix cached. L7-PA retains the highest cache hit rate throughout the sweep, but this advantage is more than offset by its queuing under load. Second, compared to the prefix-oblivious baselines, KVSwitch achieves 9.3%/59.0% lower p50/p95 TTFT than L4-ECMP with a 10.7 pp higher cache hit rate, and 12.5%/12.5% lower TTFT than L7-RR with an 8.8 pp higher hit rate.

5.4 Ablation Study

We next turn to Q2 and ablate two of KVSwitch's key design choices.

Per-prefix ECMP vs prefix pinning. To quantify the importance of per-prefix weighted ECMP, we replace it with a *pinning* policy that routes every request with a matching prefix to a single worker. At 200 rps, disabling ECMP yields a modestly higher cache hit rate (+12.7 pp), since all requests go to the single worker that caches the prefix, but inflates p95 TTFT by 2.77× (Figure 9, top). Without load spreading, a single worker becomes the bottleneck for every popular prefix. This confirms that cache locality and load balance must be jointly optimized, as argued in Section 3.4.

Impact of warm-up. KVSwitch's frequency-threshold admission requires a prefix to be observed several times before it is installed in TCAM. We measure the benefit of a warm-up phase by running the 500-request workload with and without warm-up. With warm-up, KVSwitch's p50 TTFT drops by 21.1% and its cache hit rate rises by 21.5 pp (Figure 9, bottom), consistent with the time the frequency-threshold admission needs to install popular prefixes in TCAM. Without warm-up, KVSwitch's fallback path behaves like uniform ECMP until admission catches up with the workload. We enable warm-up in the remaining end-to-end and sensitivity experiments to report steady-state behavior.

5.5 Sensitivity Analysis

To address Q3, we close by studying KVSwitch's sensitivity to three parameters: the prefix sharing ratio, the per-worker KV cache capacity, and the number of workers.

Prefix sharing ratio. Figure 10 sweeps the prefix sharing ratio from 0.2 to 0.8 at 10 rps with no warm-up. KVSwitch's cache hit rate is consistently higher than L4-ECMP and L7-

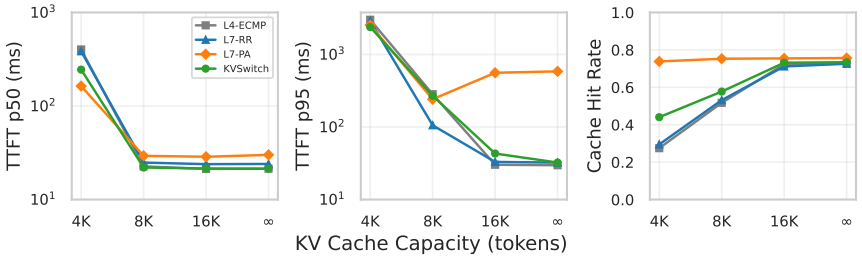


Figure 11: Sensitivity to KV cache capacity, at 200 rps, 500 requests, 0.8 prefix sharing ratio, with warm-up.

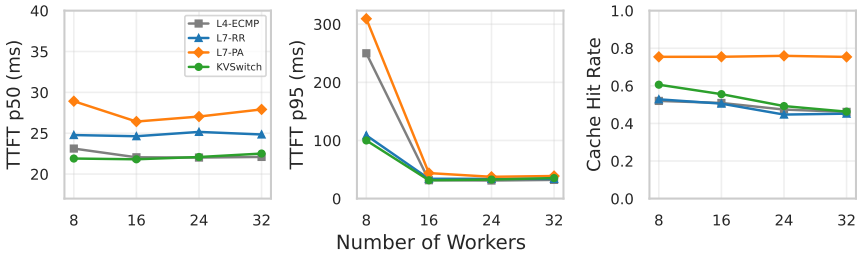


Figure 12: Sensitivity to the number of workers, at 200 rps, 500 requests, 0.8 prefix sharing ratio, with warm-up.

RR across the full range, and its p50 TTFT is consistently lower than L7-RR and L7-PA. The gap to L7-PA's cache hit rate narrows at high sharing ratios as L7-PA's software matching benefits from the more predictable access pattern, yet KVSwitch retains a clear TTFT advantage because L7-PA's proxy path remains on the critical path.

KV cache capacity. Figure 11 sweeps the per-worker KV cache capacity from 4K to 16K tokens and an uncapped setting, at 200 rps with warm-up. At low capacities, prefix-aware routing is most valuable because only a small subset of workers can hold each prefix: KVSwitch substantially reduces TTFT over both L4-ECMP and L7-RR. L7-PA underperforms at 4K because its prefix-pinning behavior magnifies queuing when cache space is scarce and hot prefixes are routed to a single overloaded worker. As capacity grows, cache hit rates saturate across all baselines and KVSwitch's relative advantage shrinks: with unlimited cache, every worker eventually holds every prefix, so prefix-aware routing adds little beyond basic load balancing.

Number of workers. Figure 12 sweeps the fabric from 8 to 32 workers at 200 rps. KVSwitch's p50 TTFT is among the lowest across the sweep: it leads at 8 and 16 workers and trails L4-ECMP by less than 0.5 ms at 24 and 32 workers. At 8 workers, KVSwitch's p95 TTFT is significantly lower than both L4-ECMP and L7-PA (100 vs 250 and 309 ms), thanks to ECMP's load spreading under tight aggregate capacity. As workers are added, TTFT differences shrink because the fabric comfortably absorbs the load, and KVSwitch's cache hit rate advantage shrinks because each prefix is less likely to see repeated requests at the same worker.

6 Limitations and Future Work

Our results rest on several assumptions that we note for full disclosure and as directions for future work.

Software prototype vs hardware switches. KVSwitch's data plane is implemented on the BMv2 software switch running inside Mininet rather than on a hardware programmable ASIC such as Tofino. Absolute and relative latency numbers will shift on real hardware, where pipeline width, TCAM size, and per-packet processing rates differ from software emulation; the qualitative gain from removing L7 routing from the critical path should nonetheless carry over. Our prototype also does not yet exercise failure scenarios: link failures, controller failover, and rule reinstallation after switch reboots are left to a production deployment.

Trace-driven simulator. Our LLM workers are trace-driven mocks that model TTFT as a linear function of cached and uncached tokens (Section 5.1). This captures the first-order effect of prefix caching on latency but not finer-grained phenomena such as batch composition effects, memory fragmentation, or tail behaviors on real GPU stacks. The profile is also drawn from a single model and serving engine (Llama-3.2-3B on vLLM v0.16.0); characteristics may differ across other models (e.g., larger dense or mixture-of-experts models), quantization regimes, and engines such as TensorRT-LLM [5] or TGI [12].

Scale. Our emulated fabric comprises a handful of spines, leaves, and workers, whereas production inference clusters serve thousands to millions of concurrent requests across much larger and more heterogeneous networks. How

KVSwitch's TCAM admission policy, controller throughput, and ECMP-group churn behave at such scale remains an open question.

Control-plane complexity and parameter tuning. Like other SDN-based systems, KVSwitch couples the network fabric, the controller, and the workers into a tightly synchronized feedback loop. Its admission threshold K, sliding window T, and probe fraction ε must be tuned to the workload: conservative choices delay the admission of popular prefixes and hurt cache hit rate, while aggressive choices admit cold prefixes and waste TCAM capacity. A systematic study of these parameters, along with adaptive tuning policies that respond to workload shifts, is future work.

7 Related Work

Distributed LLM serving and prefix-aware routing. Recent LLM serving systems have introduced various forms of prefix caching to amortize the cost of shared prompt prefixes. vLLM [20] organizes the KV cache into fixed-size blocks addressed via a hash map, enabling block-level prefix reuse within and across requests. SGLang [28] represents the cache as a radix tree and pairs it with a prefix-aware L7 router that dispatches each request to the replica holding the longest matching prefix. Parrot [21] treats the prompt as a first-class *semantic variable* and leverages this structure for workload-aware scheduling. Prefix-aware dispatching is also standard in production inference gateways such as llm-d [8] and Ray Serve's prefix-aware router [9]. These systems share our goal of maximizing prefix cache reuse, but make the routing decision in software on the critical path of every request. KVSwitch complements them by offloading the dispatch itself into programmable switches, eliminating the L7 router's per-request overhead while preserving prefix-aware placement.

Congestion-aware and weighted multipath routing. Beyond plain ECMP, a line of work incorporates path-level load signals into in-network routing. CONGA [13] measures per-leaf round-trip utilization and adjusts flow placement to avoid congested paths, while WCMP [29] assigns bandwidth-proportional weights to compensate for asymmetric fabrics. These schemes target path congestion rather than server-side load and remain oblivious to application-level state. KVSwitch's per-prefix weighted ECMP similarly uses load signals, but the signal is worker-side (queue length and batched token budget) and the weights are scoped per prefix to navigate the tradeoff between cache reuse and load balance. The two lines of work are complementary: CONGA-style path telemetry could feed KVSwitch's ECMP weights in deployments where path congestion is non-negligible.

Application-aware in-network load balancing. A growing body of work pushes application semantics into programmable switches so that routing decisions can run at line rate. AppSwitch [15] performs L7 load balancing for key-value traffic within a software switch, while LBAS [27] extends this direction by taking server-side state into account when steering web requests. R2P2 [19] raises the abstraction one level and treats RPCs as first-class datacenter citizens routed at the switch. NetCache [18] and DistCache [22] go further and use on-switch SRAM to cache hot keys directly in the fabric, both load-balancing across backends and serving requests from the switch itself. KVSwitch is in the same family but targets a different application structure: instead of matching on an opaque key or a single-level RPC, it routes on hierarchical prefix hashes over a TCAM pipeline that mirrors the chunk-structured semantics of the LLM KV cache, and it co-designs a cache-event-driven control plane that keeps data-plane rules synchronized with the distributed, continually shifting GPU cache.

8 Conclusion

Centralized L7 routers in distributed LLM serving introduce significant overheads on TTFT because every request must traverse a software proxy that tokenizes, matches, and re-dispatches its payload. To eliminate this overhead, we presented KVSwitch, a system that offloads prefix-aware routing into the network fabric itself. A client SDK annotates each request with a hierarchical prefix hash chain in a compact shim header. Programmable switches perform hierarchical TCAM matching and per-prefix weighted ECMP routing at line rate, and a cache-event-driven SDN controller keeps the switches' forwarding state synchronized with the distributed GPU cache state. On a BMv2-based prototype driven by an LLM serving simulator, KVSwitch achieves up to 27% lower median TTFT and 76% lower tail TTFT relative to a state-of-the-art L7 prefix-aware router, while preserving load balance under TCAM capacity constraints.

Acknowledgments

We thank our CS6204 instructor, Professor Richard Ma, and teaching assistant, Tiantong Hu, for an engaging course and their helpful feedback throughout this project. During the preparation of this work, we used AI coding assistants, in particular Anthropic's Claude Code, to refine the language of this manuscript and to assist with implementing the prototype. After using these tools, we reviewed and edited the content as needed and take full responsibility for the content of this work.

References

[1] anon8231489123/ShareGPT_Vicuna_unfiltered · Datasets at Hugging Face — huggingface.co. https://huggingface.co/datasets/anon8231489123/{S}hare{G}{P}{T}_{V}icuna_unfiltered.

[2] Best practices for autoscaling large language model (LLM) inference workloads with GPUs on Google Kubernetes Engine (GKE) | GKE AI/ML | Google Cloud Documentation — docs.cloud.google.com. https://docs.cloud.google.com/kubernetes-engine/docs/best-practices/machine-learning/inference/autoscaling.

[3] ChatGPT — chatgpt.com. https://chatgpt.com/.

[4] Claude — claude.ai. https://claude.ai/.

[5] GitHub - NVIDIA/TensorRT-LLM: TensorRT LLM provides users with an easy-to-use Python API to define Large Language Models (LLMs) and supports state-of-the-art optimizations to perform inference efficiently on NVIDIA GPUs. TensorRT LLM also contains components to create Python and C++ runtimes that orchestrate the inference execution in a performant way. — github.com. https://github.com/{N}{V}{I}{D}{I}{A}/{T}ensor{R}{T}-{L}{L}{M}.

[6] GitHub - vllm-project/vllm at v0.16.0 — github.com. https://github.com/vllm-project/vllm/tree/v0.16.0.

[7] Google Gemini — gemini.google.com. https://gemini.google.com/.

[8] llm-d - Kubernetes-Native Distributed LLM Inference with vLLM | llm-d — llm-d.ai. https://llm-d.ai/.

[9] Prefix-aware routing — Ray 2.55.0 — docs.ray.io. https://docs.ray.io/en/latest/serve/llm/user-guides/prefix-aware-routing.html.

[10] Scaling LLMs with NVIDIA Triton and NVIDIA TensorRT-LLM Using Kubernetes | NVIDIA Technical Blog — developer.nvidia.com. https://developer.nvidia.com/blog/scaling-llms-with-nvidia-triton-and-nvidia-tensorrt-llm-using-kubernetes/.

[11] Scaling your LLM inference workloads: multi-node deployment with TensorRT-LLM and Triton on Amazon EKS | Amazon Web Services — aws.amazon.com. https://aws.amazon.com/blogs/hpc/scaling-your-llm-inference-workloads-multi-node-deployment-with-tensorrt-llm-and-triton-on-amazon-eks/.

[12] Text Generation Inference · Hugging Face — huggingface.co. https://huggingface.co/docs/text-generation-inference/.

[13] Mohammad Alizadeh, Tom Edsall, Sarang Dharmapurikar, Ramanan Vaidyanathan, Kevin Chu, Andy Fingerhut, Vinh The Lam, Francis Matus, Rong Pan, Navindra Yadav, and George Varghese. Conga: distributed congestion-aware load balancing for datacenters. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, SIGCOMM '14, page 503–514, New York, NY, USA, 2014. Association for Computing Machinery.

[14] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.*, 44(3):87–95, July 2014.

[15] Eyal Cidon, Sean Choi, Sachin Katti, and Nick McKeown. AppSwitch: Application-layer Load Balancing within a Software Switch. In *Proceedings of the First Asia-Pacific Workshop on Networking*, APNet '17, pages 64–70, New York, NY, USA, August 2017. Association for Computing Machinery.

[16] Pranjal Gupta, Karan Bhukar, Harshit Kumar, Seema Nagar, Prateeti Mohapatra, and Debanjana Kar. Scalable and efficient large-scale log analysis with llms: An it software support case study, 2025.

[17] Christian Hopps. Analysis of an Equal-Cost Multi-Path Algorithm. RFC 2992, November 2000.

[18] Xin Jin, Xiaozhou Li, Haoyu Zhang, Robert Soulé, Jeongkeun Lee, Nate Foster, Changhoon Kim, and Ion Stoica. Netcache: Balancing key-value stores with fast in-network caching. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, page 121–136, New York, NY, USA, 2017. Association for Computing Machinery.

[19] Marios Kogias, George Prekas, Adrien Ghosn, Jonas Fietz, and Edouard Bugnion. R2P2: Making RPCs first-class datacenter citizens. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 863–880, Renton, WA, July 2019. USENIX Association.

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery.

[21] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient Serving of LLM-based Applications with

Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, Santa Clara, CA, July 2024. USENIX Association.

[22] Zaoxing Liu, Zhihao Bai, Zhenming Liu, Xiaozhou Li, Changhoon Kim, Vladimir Braverman, Xin Jin, and Ion Stoica. DistCache: Provable load balancing for Large-Scale storage systems with distributed caching. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 143–157, Boston, MA, February 2019. USENIX Association.

[23] Donald R. Morrison. Patricia—practical algorithm to retrieve information coded in alphanumeric. *J. ACM*, 15(4):514–534, October 1968.

[24] Alberto Sánchez Pérez, Alaa Boukhary, Paolo Papotti, Luis Castejón Lozano, and Adam Elwood. An LLM-based approach for insight generation in data analysis. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 562–582, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics.

[25] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In D. Song, M. Carbin, and T. Chen, editors, *Proceedings of Machine Learning and Systems*, volume 5, pages 606–624. Curan, 2023.

[26] Junyi Ye, Mengnan Du, and Guiling Wang. DataFrame QA: A universal llm framework on dataframe question answering without data exposure. In Vu Nguyen and Hsuan-Tien Lin, editors, *Proceedings of the 16th Asian Conference on Machine Learning*, volume 260 of *Proceedings of Machine Learning Research*, pages 575–590. PMLR, 05–08 Dec 2025.

[27] Jiao Zhang, Shubo Wen, Jinsheng Zhang, Hua Chai, Tian Pan, Tao Huang, Linquan Zhang, Yunjie Liu, and F. Richard Yu. Fast Switch-Based Load Balancer Considering Application Server States. *IEEE/ACM Transactions on Networking*, 28(3):1391–1404, June 2020.

[28] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2025. Curran Associates Inc.

[29] Junlan Zhou, Malveeka Tewari, Min Zhu, Abdul Kabbani, Leon Poutievski, Arjun Singh, and Amin Vahdat. Wcmp: weighted cost multipathing for improved fairness in data centers. In *Proceedings of the Ninth European Conference on Computer Systems*, EuroSys '14, New York, NY, USA, 2014. Association for Computing Machinery.

A Reproducibility

The source code, pre-compiled P4 artifacts, experiment scripts, and analysis notebooks for KVSwitch are publicly available at https://github.com/kaiitunnz/kvswitch. This appendix documents how to reproduce the results reported in Section 5 and how to run ad-hoc evaluations against the Mininet-emulated fabric.

Prerequisites. Experiments run inside a pre-built Docker image that bundles Mininet, BMv2, and all Python dependencies, so no local installation is required beyond Docker with `--privileged` support. Two data artifacts must be obtained out-of-band before the first run.

First, the ShareGPT conversations dataset, available from HuggingFace at https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered, must be placed at `data/ShareGPT_V3_unfiltered_cleaned_split.json` in the repository root.

Second, the `meta-llama/Llama-3.2-3B-Instruct` tokenizer must be pre-downloaded to the host, since the container runs with `TRANSFORMERS_OFFLINE=1` and cannot reach HuggingFace at runtime. The model is gated; access must first be requested at https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct. Once approved, install the HuggingFace CLI on the host, log in with an access token (from https://huggingface.co/settings/tokens), and download the tokenizer files:

```
curl -LsSf https://hf.co/cli/install.sh | bash
hf auth login
hf download meta-llama/Llama-3.2-3B-Instruct \
  --include "tokenizer*" "*.json"
```

The harness bind-mounts the host's HuggingFace cache into the container; it uses `$HF_HOME` if set and otherwise defaults to `~/.cache/huggingface/`. Verify that the downloaded tokenizer files appear under that cache directory (e.g., `~/.cache/huggingface/hub/models--meta-llama--Llama-3.2-3B-Instruct/`) before running any experiment.

The compiled P4 artifacts are committed to the repository under `build/p4/kvswitch/`, so no P4 toolchain is required.

Reproducing the results in Section 5. The full experiment suite is driven by a single script:

```
bash exp/run_exp.sh
```

This runs all seven experiments in sequence: the microbenchmark (Section 5.2), the end-to-end rate sweep (Section 5.3), two ablation studies (Section 5.4), and three sensitivity studies (Section 5.5). Raw traces, logs, and regenerated figures are written to `results/exp/`. Individual experiments can also be invoked by ID, e.g., `bash exp/run_exp.sh 2 4b`; IDs `1`, `2`, `3a`, `3b`, `4a`, `4b`, and `4c` correspond to the experiments listed above in order.

Ad-hoc evaluation runs. To explore parameter settings beyond those reported in Section 5, the underlying `run_eval.sh` harness can be invoked directly. It accepts a list of baselines and workload parameters such as the number of requests and the request rate. For example:

```
bash exp/run_eval.sh \
  --baselines l4_ecmp,l7_rr,l7_pa,kvswitch \
  --num-requests 200 --request-rate 10
```

Each invocation spins up a fresh Mininet instance and runs the selected baselines in turn. Pass `--build` to rebuild the Docker image if the P4 program or harness has been modified.

Pre-computed results. To skip the re-execution step, the paper's raw results, profiling traces, and figures are archived at https://drive.google.com/file/d/1PXIASia686i2M-tb8rsTdVkEaQNiVlBy/view?usp=sharing. Extracting the archive into the repository root populates `results/`, after which the notebooks in `notebooks/` (notably `result_analysis.ipynb`) can be run in Jupyter to regenerate all figures reported in Section 5.